



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start!
- Let's Explore!
- Let's Practice!
- Extension Activity
- Target Words
- Learning Points

**Let's Start!**

Reading and Writing About Comments in Java

Java
/*text*/
// text.
/**documentation*/

Learning Outcome:

Understand important information about client software for the identity metasystem in spoken or written documents.

Ask your partner:

- What do you know about commenting in a programming task?
- Do you know how to use the Javadoc tool?

Can Do Statements:

- I can understand written information about the use of comments when working in Java.
- I can list the different kinds of comments to use when working with Java.
- I can write a short text to briefly explain different kinds of comments in Java.





Let's Explore!

Sara has written a text with some useful information about comments in Java.



Instructions: Read the text below.

Teacher: Good morning, everyone. Today, we'll go back over what we've studied these days on Java. Yesterday, we read about commenting. So, Sara, what can you tell me about it?

Sara: Well, I've done some research, and I've written a short text in order to organize the information I found.

Teacher: Good! Go ahead, read your text.

Sara: Ok. "Commenting is an essential component in any programming task since it helps the programmer or the one who is reading the code to fully comprehend the function of that code inside the program, not to mention the amount of time you can save by using it. These comments or notes, which are included in a Java code file, are always overlooked by the compiler and the runtime engine, so they do not affect how the program runs. One of the advantages of commenting in Java is that we can add an unlimited number of comments, so we don't need to restrict the notes. However, to do it properly, we've got to know certain rules and types of comments that will let us organize the information, so we can better the readability of the code. First, we should know that Java supports three types of comments: Single Line Comment, Multi Line Comment, and Documentation Comment. Each one of these comments allows you to add your comments based on their length and intention. Single line comments, for example, are known as the simplest ones since they are used to add information that does not extend one single line. Start typing two forward slashes (//) and then your comment. Remember that you can also form nested comments, which are single line comments that are included in a block of code. The multi-line comments, also known as the traditional comments, go beyond one single line. To make use of them, enclose your comment between (/*) and (*). Finally, there are documentation comments, or doc comments, which are used to document the API of the code you're writing. To use them you should know how to use the Javadoc tool. They are very similar to the multi-line comments, but you have to place your comment between (/**) and (*)."

Teacher: Well done! I'd like to complement what you've said with some tips we reviewed the class you didn't come. First, avoid getting used to over commenting. Although comments are extremely useful, you don't need to fill the code up with them; it should flow logically without the need of a great number of comments. Second, add your comments while you're working on your source code. If you add them later, you may have lots of problems. Finally, don't forget to modify and update your comments as you move forward!

**Let's Practice!**

Comments classification.

Instructions: Read the sentences. Write in the blanks. Use: has written, noted, I learned, knew a, tried to learn, We've summarized, practiced.

1. I've read these days about comments in Java. Yesterday, I _____ they are essential in any programming task.
2. Sara _____ this summary. Here, she _____ that using comments in Java helps the programmer to fully comprehend the function of the code he is working with.
3. Last week, we bought this book on Java principles. _____ the chapter about comments, and we learned that there are three types of comments in Java: Single Line Comment, Multi Line Comment, and Documentation Comment.
4. Last week, I _____ programmer who taught me how to enter multi line comments. Since then, I've _____ the syntax of these comments by heart, but I haven't managed to do it! Do I have to enter my comment between (/*) and (*/)?
5. Last semester, I studied how to use the single line comments. I've _____ a lot, and now I am an expert on writing statements that don't extend the line!

**Let's Practice!**

Some Useful Information on Comments in Java

Instructions: Organize the mystery sentence.

1. The / go beyond /comments / multi-line / one line.

2. The/ overlook / and the /compiler/ runtime engine / your comments.

3. You / don't need / can be / your comments; / to restrict / they / unlimited



Extension Activity

Create a guide to help future developers.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Write notes and create a guide with some tips or recommendations for future programming developers.
- Include pieces of advice on how to develop comments in Java.
- Share the guide with your partner.

Self-Assessment

Yes

No

- I write notes and create a guide with some tips or recommendations for future programming developers.
- I Include pieces of advice on how to develop comments in Java.
- I share the guide with your partner.

☐ ☐☐ ☐☐ ☐

Target Words

- **restrict** (verb): to limit
e.g. One of the advantages of commenting in Java is that we can add an unlimited number of comments, so we don't need to restrict the notes.
- **go beyond** (phrasal verb): to exceed an established limit
e.g. The multi-line comments, also known as the traditional comments, go beyond one single line.
- **move forward** (phrasal verb): to advance
e.g. Finally, don't forget to modify and upgrade your comments as you move forward!
- **go back over** (phrasal verb): to review something one more time
e.g. Today, we'll go back over what we've studied these days on Java.
- **better** (verb): to improve
e.g. We've got to know certain rules and types of comments that will let us organize the information, so we can better the readability of the code.
- **overlook** (verb): to ignore
e.g. These comments or notes, which are included in a Java code file, are always overlooked by the compiler and the runtime engine.



Learning Points

Present Perfect vs. Past Simple

The present perfect and the past simple are verb tenses you use to talk about past events. Let's see how they are different.

The **present perfect** is used to talk about actions that started in the past and continue to the present. Also, to talk about an action that started in the past and whose consequence is still visible in the present. The time the action happened is unstated since it is not relevant in the context. To form the present perfect, we need the auxiliary **have/ has** and the **past participle** of the main verb. Have a look at this structure:

Subject + have/has+ verb past participle+ complement

For example:

*I **have worked** with Java for 3 years.* (The exact time I started to work with Java is not important, so it is not stated. Moreover, the action is still visible in the present since I'm still working with it.)

The **past simple**, for its part, is used to describe an event that started in the past and finished in the past. Besides, the time it happened is important in the context, so, different from the present perfect, the time is stated. Look at the structure:

Subject+ verb in past form+ complement.

For example:

*I **started** to work with Java **in 2008**.* (Note that the time is stated.)

***Yesterday**, the teacher **explained** how to use comments when working with Java.* (The action started in the yesterday and finished yesterday.)

Examples

- *Today, we'll go back over what we've studied these days on Java. Yesterday, we read about commenting.*
- *I've done some research, and I've written a short text in order to organize the information I found.*
- *I'd like to complement what you've said with some tips we reviewed the class you didn't come.*





Learning Points

Examples of Comments in Java

Java comments are statements or notes that are included in the Java code. The compiler and the runtime engine don't execute the comments since they are only there to provide information or to explain something about the variable, class or method in which they are included.

Java supports 3 types of comments: Single Line Comment, Multi Line Comment and Documentation Comment.

Single Line Comment

To make use of a single line comment, start typing two forward slashes (//) and then your comment:

```
// this is my comment
```

The compiler identifies immediately that all the information next to these slashes is a comment.

Multi Line Comment

This is also known as a traditional comment. To make use of it, write your comment enclosing it between (/*) and (*/*).

```
(/*) This is my very first  
comment (*/*)
```

As you can see, this type of comment is used when the information you want to enter does not meet the requirement of a single line.

Documentation Comments

You use this kind of comment to document when you want to document the API of the code you're forming. In order to accomplish this, you'll need to use the Javadoc tool. Put the comment between (/**) and (*/*).

```
/** * This is my* comment. */
```